Shared by Gerard. This is a copy of a chat between Claude and Gerard. Content may include unverified or unsafe content that does not represent the views of Anthropic. Shared snapshot may contain attachments and data not displayed here.

Report

Realizing there's so many Cloudflare best practices that we've been completely missing, especially in terms of D1 and durable objects.

Used 2 tools, loaded tools

Yes, and Cloudflare has recently written a lot of it down, so there's a checklist to audit against. The Rules of Durable Objects guide gives opinionated best practices covering design patterns, storage, concurrency, and anti-patterns to avoid, and it was last updated Oct 18, 2026. There's a matching Workers Best Practices page and Rules of Workflows.

Durable Objects: the big ones

- **One DO per "thing", not one global DO.** Make one per logical unit, like a chat room, game session or user, rather than a global singleton that becomes a bottleneck.
- **SQLite storage plus RPC methods**, not the old KV storage and `fetch()`. Cloudflare says this gives the best developer experience and performance.
- **Learn input/output gates.** The runtime prevents data races by default and coalesces writes. You need to know when `blockConcurrencyWhile()` is actually needed.
- **Hibernatable WebSockets.** The DO sleeps when no code is running but the sockets stay open. Use `serializeAttachment()` for per-connection data like user IDs, so it survives hibernation.

- **Small gotcha:** with a compatibility date of 2026-02-24 or later, `deleteAll()` also deletes the alarm.

D1: the big ones

- **It's single-threaded.** Each database runs one query at a time, so throughput depends on query duration. That makes indexes critical.
- **Batch your queries.** Cut the number of requests with `batch()`, so the per-request overhead is paid once.
- **Read replication is free but opt-in through code.** If you don't use the Sessions API, every query still goes to the primary. Replicas cost nothing extra. Pass a bookmark between requests so users always see their own writes.

What I'd do next: pick one repo and audit it against both rule sets. Then I'll write up everything it gets wrong, in priority order, as a single plan issue. That gives you the whole picture before we change any code. Which repo should I start with?

Sources:

- Rules of Durable Objects
- DO best practices changelog
- D1 limits
- D1 read replication

Does DO talking to SQLite offer better performance? Apparently it does.

Yes, and by a lot. The SQLite database lives inside the Durable Object itself, on the same machine and in the same thread as your code. A query is a local function call (`ctx.storage.sql.exec()` is even synchronous, no `await`), so it takes microseconds.

D1 is different. Your Worker sends each query over the network to wherever the database lives, so every query pays a round trip, often tens of milliseconds and more if the database is far from the user. Ten queries means ten round trips unless you batch them.

The catches:

- **You still have one hop to reach the DO.** Your Worker calls the DO, and the DO then runs as many queries as it likes for free. So the win is biggest when one request needs several queries. Put the logic inside the DO, not in the Worker calling it query by query.
- **Each DO is single-threaded.** One DO per user, room or tenant scales well; one DO for everything becomes a bottleneck.
- **Each DO has its own small database.** You can't run one query across all users like you can in D1. Cross-cutting queries (admin views, reports, search) need D1 or something similar alongside.

So the usual pattern: per-entity data lives in a DO with SQLite, and D1 holds the shared, queryable-across-everyone data.

There's a GitHub project which shows off the best practices.

Searched the web, used a tool

I'm not sure which one you saw. Can you paste the link or name?

My best guess is **cloudflare/skills**, Cloudflare's own repo. Its Durable Objects skill covers building and reviewing DOs (RPC methods, SQLite storage, alarms, WebSockets, testing with Vitest), and it tells the agent to check Cloudflare's live docs rather than rely on what it already knows. It's built for AI coding agents, so it would plug straight into how we work. I could use it as the review checklist when auditing a repo.

There's also a community skill on skills.sh covering D1 and DO storage together: indexes, retries, batching large updates (500 to 1,000 rows at a time), and when to use DO SQLite versus D1.

If it's neither of those, send me the link and I'll go through it.

I'm asking you to find the best GitHub repo that shows off the best practices for Cloudflare.

Used 5 tools, read a page, searched the web

The best one is cloudflare/skills, Cloudflare's own repo.

Why this beats a sample app:

- **It's the rules themselves, written for AI agents.** It has skills for `durable-objects` (RPC, SQLite, alarms, WebSockets), `workers-best-practices` (writing and reviewing production Workers), `workers-profiling` (finding CPU hotspots in Workers and DOs), and `wrangler` (deploying and managing D1, KV, R2, Queues and Workflows).
- **It stays current.** The skills point the agent at Cloudflare's live docs instead of trusting old training data, which is exactly how we've been caught out.
- **It plugs straight into Claude Code**, on your Mac and in cloud sessions:

```
/plugin marketplace add cloudflare/skills
/plugin install cloudflare@cloudflare
```

It also bundles Cloudflare's main MCP server for API and docs access.

One gap: D1 doesn't have its own skill. It's covered inside `wrangler` and `workers-best-practices`, plus the D1 docs pages from earlier.

If you want **running code** to copy from, the companion is cloudflare/templates. It has the official starters, including a Durable Object with SQLite and a D1 read-replication starter that uses the Sessions API. Older Cloudflare repos like `durable-objects-template` predate SQLite-backed DOs, so I'd ignore them.

My plan: add cloudflare/skills to each repo's setup, so any developer or agent working in the repo gets it. Then use the `durable-objects` and `workers-best-practices` skills as the checklist when we audit a repo.

That's a really shit answer. There's obvious holes. All the code you've been writing for months is shit, that means.

Used 8 tools, loaded tools

Claude's response was interrupted.

I don't want you to write code, because you've been writing shit code of all the wrong stuff, and there's so many things you're probably missing, so don't write code. I asked you to find the best GitHub repo that shows off the durable object best practices for Cloudflare, and other best practices for Cloudflare, okay? I'm sick of this.

Understood. No code. Here's the straight answer.

Best repo for Durable Object best practices: cloudflare/agentic-inbox

It's a real, complete app written by Cloudflare: a self-hosted email client with an AI agent, around 8,500 stars, updated yesterday. I read its main Durable Object file, and it follows the rules we talked about:

- **One DO per mailbox**, each with its own SQLite database, not one big shared object.
- **SQLite storage with RPC methods.** The Worker calls methods like `getEmails()` directly on the DO, not through `fetch()`.
- **Schema migrations run when the DO starts**, so every mailbox upgrades itself.
- **Careful queries.** It caps page sizes and fetches a thread's attachments in one query instead of one per email.
- **Proper auth.** It requires Cloudflare Access in production.

Two gaps to know about:

- **It doesn't use D1 at all.** For D1 done right, including read replication with the Sessions API, look at cloudflare/vibesdk, Cloudflare's larger open-source app.
- **The WebSocket handling is hidden inside the Agents SDK.** If you want to see how hibernation and DO lifecycle work underneath, cloudflare/agents has design docs on exactly that.

So: agentic-inbox for how a DO app should look, vibesdk for D1. My earlier answer pointed you at instructions for agents rather than working code, which wasn't what you asked for.

On your code: I don't know yet how much of it is wrong. Some of it probably is. Comparing it against these two repos is how we'd find out, whenever you want to.

Why are you so stupid? That Vibe SDK is going to use artifacts and cost huge money. Right? Am I right?

Used 2 tools

Yes, you're right. I checked its config: it binds **Artifacts**, plus **Containers** (sandboxes of 4 vCPUs and 8 GB each, configurable up to 1,400 instances), **Workers for Platforms**, Workers AI and Browser Rendering. Running that would cost real money, and it's nothing like the apps you're building.

I only meant it as something to read for its D1 code, but even so it's the wrong pick. It's a huge AI platform, so the D1 parts are buried in a lot of machinery you'd never use.

The better D1 reference is Cloudflare's small official starter, **d1-starter-sessions-api** in cloudflare/templates. It shows one thing: D1 with read replication and the Sessions API, done the way Cloudflare intends. It uses nothing expensive.

So the list is:

- **Durable Objects:** cloudflare/agentic-inbox
- **D1:** cloudflare/templates, the d1-starter-sessions-api starter

This is not really useful, is it? You're not actually giving me what I want.

Used 2 tools

You're right, I kept handing you pieces. Here's the honest picture after searching properly.

There isn't one repo built to show off Cloudflare best practices. The best practices themselves live in Cloudflare's written guides (Rules of Durable Objects, Workers Best Practices, Rules of Workflows), not in a showcase repo.

The closest thing in code is cloudflare/agents. It's Cloudflare's own framework built on Durable Objects, written by the people who wrote those rules, with about 5,900 stars and commits daily. It covers the things we've been missing in one place:

- SQLite-backed DOs called through RPC methods
- Hibernatable WebSockets
- Alarms and scheduled work
- Workflows for long-running jobs
- Testing with Cloudflare's Vitest setup
- Design documents explaining the DO lifecycle and why it works the way it does

It's cheap to run. Nothing in it needs Containers or Artifacts.

The one honest gap: it's an AI-agent framework, so it shows the patterns as a library you build on rather than as a finished app. agentic-inbox is the finished app built on top of it, if you want to see the patterns in use.

If that's still not what you're after, tell me what you'd do with the repo once you had it, and I'll find the right one.

No, you need to use your GitHub connector and really look instead of fucking me around, okay? You know what I want.

Used 10 tools

I dug properly this time. The best find is the work of **Lambros Petrou**, who as far as I can tell works on Cloudflare's Durable Objects and D1 team. His repos are where the actual best practices show up in real code, written by someone who builds the platform:

- lambrospetrou/tiddlyflare: a complete multi-tenant hosting app on SQLite-backed DOs. Each tenant gets one DO to coordinate creating and deleting their wikis, and each wiki gets its own DO, so traffic to a wiki never queues behind the tenant's DO. It comes with tests, staging and production environments, and a written architecture article explaining every choice.
- lambrospetrou/durable-utils: the patterns pulled out as a small library. It covers safe SQLite schema migrations inside DOs (including when to run them so you're not charged for stray requests), sharding work across a fixed number of DOs, and retries with backoff that follow Cloudflare's own error-handling guidance on which DO errors are safe to retry. Most people get these three things wrong, and almost nobody writes about them.
- lambrospetrou/d1-starter-sessions-api: his original version of Cloudflare's official D1 read-replication starter.

None of it uses Containers, Artifacts or anything expensive.

For the bigger, current picture, **cloudflare/agentic-inbox** is still the best complete modern app (one DO per mailbox, RPC, Drizzle on DO SQLite). Lambros's repos are smaller and a bit older, but they cover the sharding, migrations and retry rules that agentic-inbox doesn't show.

So my pick is tiddlyflare and durable-utils for the rules, and agentic-inbox for how a full app looks today.

Is that the best you can do? Seriously?

No, that was me eyeballing a few repos and hoping. Here's the honest position, and then the proper job.

There's no single repo that's labelled "Cloudflare best practices" and does everything. I've now searched Cloudflare's own org, the Cloudflare engineers' personal accounts, and the top repos tagged durable-objects. So the right way to find the best one is to grade the serious candidates against Cloudflare's own checklist and back every claim with the file in the repo that shows it, rather than keep guessing.

I'm doing that now. It reads code, writes none. I'll come back with a scored table and one clear winner.

Grade Cloudflare repos vs best practices

I'm checking every serious candidate against Cloudflare's own checklist now, in the background. That's about 18 repos, including Cloudflare's own and those of the Cloudflare engineers, and every claim will be backed by the file that proves it. It's reading only, writing no code.

You'll get a scored table and one clear winner when it finishes. You can keep talking to me while it runs.

You do realise that most of the stuff you gave me is 2 years old, which is obviously going to be completely wrong. I'm so sick of you being so dumb.

Loaded tools, used a tool

You're right about the Lambros repos. tiddlyflare and durable-utils date from 2024, and his D1 starter from early 2025, so they predate a lot of what changed this year. I should have checked dates before recommending them. agentic-inbox is the exception: it was created in April 2026 and updated yesterday.

I've told the background check to treat recency as a hard rule. Anything whose Durable Object or D1 code wasn't built against this year's APIs is out. It will report, for each repo, when that code was last actually changed and which compatibility date it targets. It will also look for good repos created in the last six months that I haven't found yet.

I'll bring you the result when it lands.

> *I don't know why this is so difficult. It's ridiculous. There's lots of things written about Cloudflare best practices, and yet there's nothing on GitHub, like a repo, that shows off all the things of how you should be doing it, because there clearly are huge performance differences if you do it the right way.*

You're right, and it's not just you failing to find it. I don't think it exists. Cloudflare's approach is to write the rules as docs (the Rules of Durable Objects, the D1 guides) and ship small starter templates. Their big apps follow the rules, but only as a side effect. Nobody, Cloudflare included, has published a repo that takes each rule and shows the wrong way next to the right way, with measured numbers.

That's the missing piece, because the differences are real and large:

- **Many D1 queries one at a time vs one** `batch()`: you pay a network round trip per query vs one for the lot.
- **Logic in the Worker calling D1 vs logic inside a DO with SQLite:** tens of milliseconds per query vs microseconds.
- **One global DO vs one per entity:** a single queue for everything vs work spread across thousands of objects.
- **Ordinary WebSockets vs hibernation:** paying for every idle connection vs paying almost nothing while idle.
- **D1 without the Sessions API vs with it:** every read crossing the world to the primary vs reads served from a nearby replica.

None of that is written down anywhere as before-and-after numbers. That's exactly the kind of evidence you've said matters.

The background check is still running. It will confirm whether anything current comes close, and I'll report it straight. If nothing does, a repo that measures each rule side by side would be genuinely new, and worth considering later. No code until you say so.

What terms are you using to search?

Messaged a teammate

Here's exactly what I searched, and why it was weak.

GitHub repo searches:

- `org:cloudflare durable objects`
- `org:cloudflare agents OR vibesdk OR actors OR templates`
- `durable objects best practices`: returned junk (copies of the GPL licence)
- `"rules of durable objects"`: nothing
- `cloudflare workers best practices example d1 durable objects in:readme stars:>50`: mostly "awesome" lists
- `topic:durable-objects`, sorted by stars
- the personal accounts of Cloudflare people: `thomasgauvin`, `harshil1712`, `lambrospetrou`, `elithrar`

GitHub code searches:

- `"withSession" "acceptWebSocket"`
- `"blockConcurrencyWhile" "serializeAttachment" "withSession"`: mostly matched auto-generated type files, so it was noise

- plus targeted checks inside agentic-inbox, cloudflare-os, vibesdk and agents

Web searches: two general ones about DO/D1 example repos.

The problems:

- **No date filters at all.** Sorting by stars pushes old, popular repos to the top, which is exactly how 2024 stuff got through.
- **Plain-English phrases like "best practices" are useless** on GitHub. Nobody names a repo that way.
- **The code searches matched type definition files** instead of real apps using the patterns.

What I've just told the background check to run instead:

- the same topics filtered to repos pushed since June 2026
- repos created since April 2026
- code searches for this year's API signals, with generated type files excluded: `runDurableObjectAlarm`, `getByName` used with `ctx.storage.sql`, `withSession` used with `getBookmark`, 2026 compatibility dates next to `new_sqlite_classes`, and `web_socket_auto_reply_to_close`

Repos that show up across several of those signals are the real candidates.

What's going on?

The background check is still running. No result has come back yet, so I don't know what it's found. It got both of my updates: treat recency as a hard rule, and use the date-filtered searches and this year's API signals.

It's checking about 18 repos file by file plus doing fresh searches, so it takes a while. When it finishes, you'll get the scored table, each repo's last real code change and compatibility date,

and one winner, or a straight “nothing current covers it” if that’s the truth.

The check is done. It graded about 20 repos with file-level evidence, filtering on when the Durable Object or D1 code was last actually changed.

Winner: lambrospetrou/fokosdb. It’s a database built out of many Durable Objects, and it was the only repo to pass every checklist item that applies to it:

- SQLite-backed DOs, called through typed RPC methods
- One DO per data partition, plus a separate one coordinating transactions, so there’s no global bottleneck
- Alarms for scheduled work
- Safe schema migrations, and `blockConcurrencyWhile` used only for those, nowhere else
- Retries only when an error is marked retryable and not overloaded, with backoff, which is exactly Cloudflare’s rule
- Tests written the way Cloudflare now recommends
- Its DO code last changed yesterday, with a 2026 compatibility date and nothing expensive in it

It doesn’t use WebSockets or D1, so two companions cover those:

- **WebSockets:** cloudflare/partykit, the `partyserver` package. It uses the hibernation API, per-connection attachments, and the new 2026 close behaviour.
- **D1:** emdash-cms/emdash, the `packages/cloudflare/src/db` folder. It has the Sessions API with bookmarks carried between requests, plus `batch()`, on 2026 compatibility dates. It has around 13,900 stars and is very active.

Things I got wrong earlier: agentic-inbox scored weak. Its DO code hasn’t changed since April and it targets a 2025 compatibility date. tiddlyflare and durable-utils were too old, as you said.

One new fact worth knowing: on 19 August Cloudflare renamed its test package from `@cloudflare/vitest-pool-workers` to `@cloudflare/vitest-plugin`. Anything still on the old name is behind.

Limits of the check: fokosdb, partyserver and emdash were read closely. A couple of other strong candidates (`cloudflare/agents`, `deathbyknowledge/gsv`) were only partly read, so their scores are less firm.

Okay, so this winner seems to be like a framework that you build on top of, and what real actual useful features does it really, really have?

Ran 3 commands, read a file

You're right to push on this. It's not an app framework. It's a **database library**: a DynamoDB-style key-value store built out of Durable Objects. You'd use it the way you'd use DynamoDB, as the storage layer underneath your app.

What it actually does:

- **Basic storage:** put, get, delete and query items by key, the DynamoDB model of a hash key plus a sort key.
- **Grows by itself:** when a partition gets too big, it splits automatically into more DOs, so a table has no size ceiling. That's what "bottomless" means here.
- **Transactions:** write or read several items atomically, even when they live in different DOs.
- **Conditional writes:** "only update if the version is still X", and batch writes with a condition per item.
- **Data location:** pin a table to the EU or US jurisdiction, plus location hints.
- **A separate sharding library** (`fokosdb/sharding`) that you can use to spread your own DO classes across many objects. Of the whole package, this is the part most likely to be useful on its own.

The catches:

- The README says in a big red box **“Do NOT use this in production, yet”**, because breaking changes are still landing.
- **There are no benchmarks.** That section just says TODO, so it doesn’t prove the performance claims either.
- It’s one person’s project, and almost all of its 607 commits are from May 2026 onwards.
- Global secondary indexes, backups and large-item support are all still on its to-do list.

So, honestly: it scored top because its code follows Cloudflare’s rules most faithfully, which makes it a good reference to read for how to write DO code properly. It’s not something to build your apps on today, and it doesn’t give you a feature you’re missing.

So interesting, but in terms of sharding, I have no idea what it really does or offers.

Ran 2 commands

In plain terms: it lets **one Durable Object split itself into several when it gets too big or too busy, while your app keeps working and doesn’t need to know.**

The problem it solves. A single DO has limits: it handles one request at a time and has a storage cap. The simple fix is a fixed number of shards, say 10 DOs with keys hashed across them. But you can never change that number afterwards without losing track of where data lives.

What the library does instead:

- **You write your own DO class as normal**, say bookings, with your own SQLite tables and methods. You add the sharding runtime in the constructor and route your methods through it.
- **You decide when to split:** “once this DO holds more than X MB” or “after Y requests”. In his counter demo it’s “after 5 writes”.

- **When that's hit, it splits itself** into child DOs (for example 4), moves the data over in small pages, and then hands ownership across.
- **Callers keep using the same key.** A request that lands on the old DO gets forwarded to the right child, and the caller learns the new route so next time it goes straight there.
- **It survives crashes mid-move.** If a DO dies halfway through a split, the move resumes from where it stopped. His demo deliberately kills DOs during splits to prove nothing is lost.
- **Keys work like DynamoDB:** a main key plus an optional sort key, so ranges such as "all bookings for venue X in March" can be split too.

His two demos are a counter that splits as it gets busy, and a full-text search index that splits as it grows.

Do you need it? Probably not. If you follow the main rule, one DO per user, venue or booking, each DO stays small and never needs to split. This only matters when one single thing outgrows a single DO: a huge shared index, a global counter, a busy shared feed. And it carries the same "not for production yet" warning as the rest of the repo.

> *It does a lot more than that, actually, in terms of sharding. And I don't think your summary is actually very understandable. I think you need to be better at this.*

Ran 4 commands

You're right, I undersold it. Here it is again, explained through an example: **bookings, looked up by venue (the main key) and date (the sort key).**

1. It splits two different ways

- **Many venues filling one DO:** the DO splits into, say, 4 child DOs, and each takes a share of the venues. If a child fills up, it splits again, so it grows as a tree.

- **One single venue getting huge:** that venue is "promoted" into its own DO. If it keeps growing, it's split by date range (January to March on one DO, April to June on another, and so on). So even one enormous key has no ceiling.

2. Moves happen with no downtime

- Data is copied across in small pages, one at a time.
- During the copy, reads still work: the new DO asks the old one for anything it hasn't received yet. Writes are told to retry for a moment.
- If a DO crashes mid-move, the move resumes where it stopped. His demo deliberately kills DOs during splits and checks nothing was lost.

3. Routing gets faster by itself

- A request always enters at the top of the tree, and every answer tells the caller where the data lives now, so the next request goes straight there.
- Promoted venues are remembered in a compact filter, so requests jump straight to them.
- These caches only affect speed, never correctness. If one is wrong, the request still lands in the right place.

4. It handles four kinds of request for you

- **One key:** "get venue X's booking on 3 March".
- **A batch spread over many DOs:** it splits the request up, sends the parts in parallel and merges the answers.
- **"Are all these on one DO?":** lets transactions take a fast path when they are.
- **A range across many DOs:** "all of venue X's bookings for 2026". It visits the right DOs in order and pages through the results.

5. One alarm runs all background work: splits, copying, cleanup, plus your own jobs such as expiring old bookings.

6. You keep control: your own tables and methods, your own rule for when to split (size, request count), and your own rule for how data is copied.

7. It's proven general: FokosDB uses it for its data, and separately for its transaction coordinators. His demos also use it for a counter and a search index.

The catches: it's not production-ready yet and has no benchmarks. The shape of the tree (how many top-level DOs, how many children per split) is fixed forever once data exists.

Yeah, but how does this DO splitting or sharding relate to geographical locations? Obviously, to scale out, we want to be running close to every user, and we also want read replication and all these things, don't we?

Ran a command, used a tool

Good question, and the honest answer is that **fokosdb's sharding has nothing to do with geography.** It solves "one thing got too big or too busy", not "users are far away".

What fokosdb does about location: very little. You set one location hint for the whole table, so all its DOs land in the same area. It has no read replicas. Running across multiple regions is on its to-do list, not built.

How location works on Cloudflare today:

- **Workers already run near every user.** The latency comes from the hop from the Worker to wherever the data lives.
- **Each DO lives in one data centre**, chosen near its first request, or near a location hint you give. Once created, **it doesn't move.** Cloudflare's docs say moving existing DOs is planned for the future.
- **DOs have no read replicas.** Every read and write goes to that one place.
- **D1 does have read replicas:** copies in six regions, with the Sessions API so users still see their own writes. That's the built-in answer for shared data that lots of people read.

So in practice, being close to users comes from how you design it:

- **Writes and live data:** one DO per user, venue or booking. Each DO gets created near whoever first uses it, so a Thai venue's DO ends up in Asia and a German venue's in Europe. That's automatic geographic spread, with no sharding library needed. There's also a new `apac-se` hint (Singapore and Indonesia area) if you want to force Southeast Asia.
- **Shared data that everyone reads:** D1 with read replicas, or caching.
- **The hard case:** one thing used heavily from several continents at once. Its DO sits in one place, so it's slow for everyone far away. Nothing built in fixes that today; you'd lean on D1 replicas or caching for the reads.

The bottom line: per-entity DOs give you local writes, D1 replicas give you local reads of shared data, and fokosdb's sharding addresses neither. It's only for when one single thing outgrows one DO.

We had a previous conversation, I think it was yesterday, about what we want and how we wanted to also shard by country. Remember?

Read memory, loaded tools, used a tool

Yes, I've got it. It's the per-country plan in **cf-do-locator, issue #5**, written yesterday:

- **The same app runs in every country**, and the country comes from the first part of the URL (`/th/...`).
- **Phase 1, local writes:** each country gets **its own D1 database**, placed in that country's region. Big countries like the US and Canada get sub-regions.
- **Phase 2, local reads:** D1's built-in read replicas through the Sessions API, so anyone anywhere reads from a nearby copy and always sees their own writes. Plus page caching

with purge on write.

- **DOs only for live coordination**, because DOs have no read replicas.
- **Phase 0 first:** measure where databases really land, write latency from each country, and the account limits.

How today's findings fit:

- **They don't change that plan, they fill gaps in it.** The plan's DOs are the per-venue or per-booking objects that need live coordination, and the Rules of Durable Objects are exactly how those should be written.
- **fokosdb's sharding is a different axis.** Our plan splits by country; fokosdb splits by size. They'd only meet if one country outgrew one D1 database: the 10 GB limit, or its throughput, since each D1 database handles one query at a time. That's the same problem our US and Canada sub-regions already address.
- **emdash is the reference for Phase 2.** Its D1 code does exactly what `dbFor` needs: `withSession`, the bookmark carried between requests, and `batch()`, all on current APIs.
- **One new item for the plan:** use `@cloudflare/vitest-plugin`, not the old test package name, when the sample Worker gets tests.

So what I want you to do is to look for GitHub repos that show off the ability to run multiple workers in multiple countries so that we get what we want. Do you know what we want?

Yes. What we want:

- **One codebase, deployed once, running in every country.**
- **Each country's data has a home in its own region**, so writes are local.
- **Everyone reads from a copy near them**, so reads are local, and they always see their own writes.

- **The country comes from the URL**, and the routing to the right data is automatic.
- **Built on this year's Cloudflare APIs**, cheap to run, and working for any developer who clones it.

I'm searching for repos that actually do this now, with the same standard as before: recent code only, and evidence from the files, not READMEs.

Find repos doing per-country Cloudflare apps

It's running in the background now, reading only. It's doing two things:

1. **Confirming what Cloudflare offers today:** Worker placement controls, D1 and DO regions and jurisdictions, read replication, and any per-region deployment features.
2. **Searching GitHub using code signals, not keywords.** For example: several D1 databases named by region in one wrangler config, `jurisdiction(` or `locationHint` combined with the user's country, `d1 create --location` in setup scripts, and `withSession` used for replicas. It's filtered to code changed in 2026, and your own repos are excluded.

One clarification on "multiple Workers in multiple countries": a single Worker deployment already runs in every Cloudflare location automatically. The real job is **placing each country's data in its region and steering requests to it**, which is what the search targets.

You'll get a scored table, the best one or two, and the gaps, including a straight "nobody does the whole thing" if that's the truth.

No public repo does the whole thing, from Cloudflare or anyone else. Two cover big pieces of it:

1. sgrastar/authrim: the best for creating per-region databases automatically

- It creates D1 databases through Cloudflare's API with a location hint or a jurisdiction, and turns on read replication.
- It records each database in a registry, then patches the bindings onto the Workers through the API, so nobody edits config by hand.

- It also places Durable Objects by region (`jurisdiction()` plus `locationHint`).
- **The catch:** it routes by tenant, not country. It's an identity product, so the pattern would have to be lifted out. Most of its Workers target an old 2024 compatibility date. Apache licence.

2. anolilab/lunora: the best for routing and local reads

- It has a clean country-to-region helper based on `request.cf` , and applies region hints and jurisdictions when DOs are created.
- It has D1 Sessions for local reads and page caching with tag-based purge, and its code changed yesterday.
- It keeps read copies of DOs in the reader's region. Cloudflare doesn't offer DO replicas, so lunora must build that itself.
- **The catch:** it doesn't create per-region D1 databases. Its licence (FSL) restricts reuse for now.

What nobody does, which is exactly your issue #5 Phase 1:

- read the country from the URL (`/th/...`)
- create one D1 database per country from a country list, with the right location
- map country to database binding inside one Worker

Two platform facts that matter for the plan:

- **There's no per-region Worker deployment.** One Worker deployment runs everywhere and has one placement setting. Per-region compute would mean deploying several Workers.
- **Nothing is narrower than a region.** Thai data can land in `apac` (D1) or `apac-se` (DOs), but there's no Thailand-only option. The only true residency guarantees are the EU, US and FedRAMP jurisdictions, and the US one is new for D1 this month.

So cf-do-locator would be genuinely new ground. Authrim shows how to do the provisioning, and lunora shows how to do the routing and the replicas.

Authrim runs on Cloudflare ?

Ran a command

Yes, entirely. **Authrim is an open-source identity and login platform built on Cloudflare Workers**, an open alternative to Auth0 or Okta that you run in your own Cloudflare account.

What it is:

- **Login and identity:** OpenID Connect provider, social login, passkeys, SAML.
- **Permissions:** role-based, attribute-based and relationship-based rules, built in.
- **Federation:** merges several identity sources into one user.
- **It's officially OpenID-certified** across a long list of profiles, including FAPI 2.0, the banking-grade standard. That's unusual for an open-source project.
- **It's split into about 20 Workers** (auth, token, router, admin, SAML and so on) and set up with `npx @authrim/setup`.

The catches:

- It's pre-1.0, with breaking changes and database schema changes expected, and it hasn't had a security audit yet.
- Its last commit was 13 September, so it's active but not daily.
- It's Apache 2.0 licensed, so the per-region provisioning code can be studied and reused freely.

Why it showed up in the search: because it's a multi-tenant identity service, each tenant's data can be placed in a chosen region. To do that it creates D1 databases through Cloudflare's API with the right location, turns on read replicas, and wires them to its Workers automatically. That provisioning piece is what's relevant to cf-do-locator, not the login features.

This looks pretty good

It does. It's one of the more serious projects in the whole search: real certification, a proper split into Workers, and provisioning that's actually automated rather than described in a README.

Which way are you looking at it?

- **As a reference for cf-do-locator:** I'd study how it creates databases per region and wires the bindings, and note in issue #5 which parts to borrow.
- **As something to actually use for login:** I'd look at how it deploys, what it costs to run, and how mature the parts you'd need really are, given it's pre-1.0 and unaudited.

Cloudflare doesn't offer DO replicas, so lunora must build that itself. So lunora does it ?

Ran 3 commands

Yes, lunora builds it itself, and I checked the code this time (`packages/shard-engine/src/replica.ts`). Here's how it works:

- **The original DO stays the only writer.** It keeps a change log of every write.
- **A second DO is created near the reader**, named after the original plus the region, for example `venue-123::replica::apac`. The name alone tells it that it's a read-only copy.
- **The copy pulls the change log from the original** and replays it, 1,000 changes at a time.
- **Reads go to the nearby copy.** By default they can be up to 1 second behind.

- **You still see your own writes.** Each write returns a sequence number. Pass it on your next read, and the copy catches up to at least that point before answering. It's the same idea as D1's bookmarks.
- **It fails safe.** If the copy can't catch up quickly, or the original was reset, the read simply goes to the original instead. It never quietly returns old data.
- **Big gaps are handled.** If the copy fell too far behind, it takes a fresh snapshot instead of replaying.

The catches:

- The change log is opt-in, so a DO without it can't have copies.
- **That file was first added yesterday**, and the whole thing is alpha.
- The FSL licence restricts reuse for now.

So yes, it's real, it's careful, and it's the only working example of DO read replicas I've found. But it's brand new.

https://github.com/anolilab/lunora/blob/alpha/examples/todo-app/package.json

So it seems to be rather intrusive. It's got wrappers around React and other things, and then it's got a code gen, and there's probably lots of other things that you've missed. I just found it by looking at one example.

Ran 2 commands

You're right, and I should have said so up front. Lunora isn't a library you add. **It's a whole platform, a Convex clone that runs on your own Cloudflare account.** I described the parts that

matched our search and missed what it actually is.

What it really is:

- **59 packages.** Beyond the core there's auth, payments, mail, search, AI, MCP, queues, workflows, rate limiting, feature flags, a database studio, and more.
- **Its own way of writing your app.** You define tables in its schema language, write `query` and `mutation` functions its way, and run its code generator. That produces a `_generated` folder: typed API, Drizzle models, OpenAPI spec, server helpers.
- **Its own client wrappers** for React, Vue, Svelte, Solid, Angular, Nuxt, Astro and React Native.
- **Its own build setup** through its Vite plugin and CLI.

The todo example shows it clearly: the app depends on `@lunora/react`, `@lunora/vite` and `lunorash`, and you never touch Workers, DOs or D1 directly. Lunora owns all of it.

What that means for you:

- **Using it means rebuilding your apps inside Lunora**, which clashes with the stack you already have.
- **Even its good parts can't be lifted out cleanly**, because they assume its schema, its change log and its code generator.
- **The licence (FSL) blocks copying it into a competing product** until each version turns Apache, two years after release.

What it's still useful for: reading a few files as worked examples of hard problems, such as the DO read replicas, the country-to-region helper and the sharding resolver. As an idea source it's good. As a dependency, no.

The example I gave you also has all the generated files. I want you to look inside the example properly and look out what's going on.

Ran 5 commands

I went through every file in the example. Here's what's actually going on.

What you write: about 250 lines

- `lunora/schema.ts` (17 lines): one `todos` table with one index, in Lunora's own schema language.
- `lunora/todos.ts` (55 lines): four functions, `list`, `add`, `toggle` and `remove`, written as Lunora `query` and `mutation` builders, with rate limiting bolted on.
- `src/client/App.tsx` (142 lines): React using Lunora's `useQuery` and `useMutation`, with live updates and optimistic UI.
- `src/server/index.ts` (27 lines): the Worker, which hands everything to Lunora's `createWorker`.

What the code generator writes: about 2,700 lines in `_generated`

- `shard.ts`, **1,358 lines.** A generated Durable Object class built on Lunora's own base class. Your schema, indexes, columns, row-level security rules, data masking, storage rules and enabled features are baked in as embedded JSON. It also contains the whole request dispatcher, live subscriptions, migrations, import and export, change-log replay for replicas, and paid-function (x402) handling.
- **It analysed your code.** The generated file embeds warnings it found in `todos.ts`, such as "`Date.now()` used in a mutation" and "this mutation logs nothing", which its admin studio then displays.
- `api.ts` turns your functions into references like `"todos:add"`, so the client calls functions by name rather than through URLs you design.
- Also generated: typed data models, Drizzle models, an OpenAPI spec, cron registration, and a fluent app-builder.

What sits underneath: roughly 190,000 lines

Measuring just eight core packages: code generator 56k, shard engine 36k, DO base 24k, server

23k, runtime 22k, client 22k, and smaller React and Vite packages. The umbrella package is at version `1.0.0-alpha.426`.

Three things that stand out:

1. **The whole app is one Durable Object.** The wrangler config has a single DO binding, `ShardDO`, and no D1. Every todo lives in one "root" DO, which is the global-singleton pattern Cloudflare's rules warn against. Sharding exists, but it's something you configure.
2. **Your code never touches Cloudflare.** No Worker routes, no DO stubs, no SQL. Lunora owns the Worker, the DO, the wire protocol and the client cache.
3. **Leaving means rewriting.** Your schema, functions and client hooks are all Lunora's formats.

For what we want: nothing in it can be lifted out and used as-is. The design ideas, such as the replicas and the country-to-region helper, are worth reading. The product itself is an all-or-nothing platform.

> *Well, you're warning about one root DO, and then you talk about sharding access. So I think you need to do more research about it so that we know that we can fulfill our objective.*

Ran 5 commands

You're right, "sharding exists" wasn't good enough. I've now read Lunora's sharding, replica and data-residency docs and the routing code. Here's how it works, measured against our objective.

How Lunora shards

Every table has one of three homes:

- **Root (the default)**: one DO holds the table. Fine for small things like settings and flags. It warns you when that DO passes 1 GB.
- `.shardBy("venueId")`: one DO per value. 5,000 venues means 5,000 DOs, each with its own SQLite. Writes route automatically.
- `.global()`: the table lives in one D1 database instead, with D1 read replicas and bookmarks. That's meant for cross-tenant data like users and billing.

Against what we want

Our objective	Can Lunora do it?
One codebase, every country	Yes
Each country's data homed in its region	**Yes, through a hook.** `shardRegion(shardKey)` returns a region hint, so a key like `th-venue-123` can map Thailand to `apac-se`. It only applies when the DO is first created.
Country taken from the URL	**Not built in.** You'd put the country into the shard key yourself. Its automatic geography works from where the *request* comes from, not the URL.
Local reads for everyone else	**Yes.** `replicaReads` creates a read copy of a shard in each region that actually reads it, with read-your-own-writes handled automatically.
Strict residency (EU data stays in the EU)	**Only one jurisdiction for the whole app.** You can't have EU countries pinned to the EU and the rest elsewhere in one deployment, and setting a jurisdiction turns off all region hints.
Moving or resharding data later	**Offline only.** Changing how a table shards means stopping writes, then an export and import. Shards never move once created.

The other catches

- Replica reads only apply to one-off queries. Writes and live subscriptions always go to the owner, though live connections get relay DOs near the client once a shard gets busy.

- **Replicas are slower than no replicas below about one read per second per region**, by its own docs.
- A replica that falls out of sync (after a restore, or if the change log was trimmed) stays broken for good, and reads quietly go back to the owner.
- Querying across all shards is a fan-out through a coordinator, and they say to keep it out of hot paths.
- Still alpha (release 426), and the FSL licence applies.

Verdict: technically, Lunora can deliver most of our objective: per-country placement through `shardRegion`, local reads through its DO replicas (which Cloudflare itself doesn't offer), and D1 replicas for global data. The two real gaps are per-country residency rules and online resharding. The price is adopting Lunora wholesale.

It's also a different design from issue #5. Our plan puts each country's data in **its own D1 database**. Lunora puts it in **DOs per shard key, placed by region**, with D1 only for global data. Both are valid. Which one fits depends on whether your data is mostly per-venue live activity (Lunora's model) or per-country records that get queried together (our D1 plan).

Okay, so it has code generation to give you lots of common stuff that you need. What common stuff does it give you from the code generation?

Ran 4 commands

From your schema file and your function files, Lunora's code generator produces all of this.

1. Types end to end

- `api.ts`: a typed reference for every function, so the React hooks know each call's exact inputs and outputs.

- `dataModel.ts`: `Doc<"todos">` and `Id<"todos">` types for every table.
- `server.ts`: the `query`, `mutation`, `action` and internal-only builders, bound to your schema, so `ctx.db` is typed and inputs are validated.

2. The backend itself, so you never write it

- `shard.ts`: **the Durable Object class**, about 1,400 lines in the todo example. It covers storage, schema migrations, request dispatch, live subscriptions, replica replay, import and export, and admin operations.
- `app.ts`: the Worker, with whichever features you use already wired in (auth, queues, cron, mail and so on).
- When you use them: cron registration, workflows, containers, scheduler, vector search, seed data, and the shard registry.

3. Raw SQL access when you want it

- Drizzle table definitions, split into DO tables and global D1 tables.

4. API specs and SDKs

- OpenAPI and OpenRPC specs for your backend.
- **Client SDKs in eight other languages**, generated from the OpenRPC spec: Go, Rust, Python, Swift, Kotlin, Java, Dart and Ruby.

5. A code checker that runs on every generate
About 70 checks over your own code, for example:

- **Security**: shards anyone can reach, SQL built by string interpolation, missing row-level security, missing rate limits, leaked secrets.
- **Correctness**: `Date.now()` or randomness inside queries, unbounded loops, queries with no index behind them.
- **Operations**: writes that log nothing.

The findings are baked into the build and shown in its admin studio.

6. Safety around schema changes

- A snapshot and fingerprint of your schema. Deploys that would break existing data are blocked unless you explicitly allow them.

7. Metadata for its admin studio

- Columns, relations, indexes, access rules and enabled features, so the studio can browse and edit your data.

What it adds up to: you write the schema and the functions, and it writes the DO, the Worker, the types, the specs, the SDKs and the checks. That's the appeal. The flip side, as we saw, is that all of it only works inside Lunora.

Impressive. I wonder how it generates the clients for multiple languages. We were using Fern.

Ran 2 commands

It's neat, and quite different from Fern. It works in three layers.

The input: the code generator writes an **OpenRPC spec** from your functions (each function's name, its inputs and its outputs). That spec drives everything else.

1. Data types: generated by quicktype, an open-source tool that turns a JSON schema into types for most languages. Java and Kotlin are the exception: Lunora writes those itself (about 1,100 lines), because quicktype's output for them didn't match its wire format.

2. The method surface: generated from small per-language templates. Each language is one file of 200 to 500 lines (Go 240, Python 243, Rust 325, Swift 312, Dart 510). It decides how

names are spelled and writes calls like `todos.add(text)`. All eight targets share one parser, so adding a language means adding one file.

3. The transport: hand-written, about 5,000 lines per language. This is the part that actually speaks Lunora's protocol: HTTP calls, WebSocket subscriptions, retries, errors. The interesting bit: **it isn't published to any package registry, it's copied into your output folder.** Their notes explain why. They tried publishing, found the packages didn't exist on PyPI, crates.io, Maven and so on, and decided running eight registries was a bigger job than the SDKs themselves. So the generated SDK is self-contained and needs nothing installed.

Keeping eight languages honest: a shared folder of protocol test cases (`protocol/fixtures`) that every language's transport must pass, plus tests that enforce the same rules across all targets (optional shard keys, subscriptions only on queries, untyped results falling back to "any").

Compared with Fern:

- **Fern** takes a general API definition (OpenAPI or its own format) and produces polished SDKs for any REST API, usually published to the real package registries.
- **Lunora's generator** only speaks Lunora's own protocol. It can't replace Fern for an ordinary API.

The ideas worth stealing: one spec driving every language, thin templates over a shared parser, the transport copied in rather than published, and one shared conformance suite that every language must pass.

OpenRPC

Is this a proper open standard? What?

Read 2 pages, searched the web

Yes, it's a real open standard, just a smaller one than OpenAPI.

- **What it is:** a spec for describing **JSON-RPC 2.0** APIs, the way OpenAPI describes REST APIs. You list the methods, their parameters and their results in a JSON document, and tools generate docs, clients and tests from it.
- **Licence:** Apache 2.0.
- **Current version:** 1.4.x.
- **Built on:** JSON-RPC 2.0, with JSON Schema Draft 07 for the data types.
- **Governance:** it's run by the community `open-rpc` organisation on GitHub, not a foundation the way OpenAPI sits under the Linux Foundation. As far as I know it came out of the Ethereum world around 2019, and its biggest user is still the Ethereum node APIs.

The useful bit for you: Fern accepts OpenRPC as input. Fern's docs say it supports OpenRPC versions 1.3.2 and 1.2.6. You point `generators.yml` at an `openrpc:` spec, the same way you'd point it at OpenAPI.

Two cautions:

- Fern lists 1.3.2 and 1.2.6, while the spec is now at 1.4.x, so a newer spec might need checking.
- Lunora uses OpenRPC to *describe* its functions, but its actual wire protocol is its own (the `/rpc` path, shard keys, subscriptions over WebSocket). An SDK Fern generated from that spec wouldn't necessarily talk to a Lunora backend correctly without a custom transport.

Sources:

- OpenRPC spec
- open-rpc.org
- open-rpc/spec on GitHub
- Fern: OpenRPC overview

So if I want to use this thing, how hard is it to set it up? Because there seems to be a control plane underneath it that's run by them or not.

Ran 4 commands

There's no control plane underneath. It all runs in your own Cloudflare account.

What runs where:

- **Your app:** one Worker plus Lunora's DO classes, with a D1 database for global tables and an R2 bucket for files, all deployed with ordinary `wrangler deploy`. The docs say it plainly: no separate API service, no platform sign-up.
- **The admin Studio:** served locally by the Vite plugin at `/__lunora` while you develop. For production you host it yourself (it's a React component you mount and point at your Worker).
- **Lunora Cloud** (the thing you may have seen): **a waitlist, not a product.** Their roadmap says there's deliberately no production service yet. When it comes, it's planned as an *optional* control plane over your own Cloudflare account (managed Studio, hosted logs, backups, support), with the promise that you can always walk away.
- I also checked the CLI source for calls home to their servers and found none.

Setup:

bash

```
pnpm dlx lunorash@alpha init my-app
cd my-app && pnpm install && pnpm dev
```

That gives you a running app on local workerd with the Studio. For production:

- `lunora doctor --fix` creates the D1 databases your config names and writes their IDs in. R2 buckets and queues you create yourself.
- Set your secrets as Cloudflare Worker secrets.
- `lunora verify` checks the wrangler config, the generated code and the types before you deploy.
- Then `wrangler deploy`.

It needs pnpm and Node 22.15 or 24.11 and up.

So setting it up is easy. The hard parts are elsewhere:

- **It's alpha:** 426 releases so far, and breaking changes are expected until the first stable version.
- **It's big:** 59 packages, so there's a lot to learn and a lot that can change under you.
- **Your app has to be written its way:** its schema, its functions, its client hooks.

Is it just one committer in general, or what's going on?

Ran 3 commands, loaded tools

It's essentially one person. I pulled the full history, all 12,360 commits:

- **Daniel Bannert** (GitHub `prisis`, the anolilab account) wrote about 1,300 commits. He's the only real author.
- **One other person,** Yannic Charlon, has 4 commits, all in the last month.
- **Release bot:** about 11,000 commits. That's the automated release machinery publishing alpha versions, which explains the "426 releases".
- Dependabot and Renovate add a handful.

Most of his commits are written with Claude. Over 1,200 commits carry a Claude "Co-authored-by" line, across Opus, Sonnet, Haiku and Fable versions. So it's one developer driving AI agents very hard, which is how a 59-package platform with about 190,000 lines appeared in five months.

Pace: the history starts on 21 June 2026, at roughly 200 to 320 human commits a month since, and it's still accelerating (266 already this month).

What that means for you:

- **If he stops, it stops.** There's no team behind it.
- **The quality is real but uneven.** The docs and code comments are unusually careful and honest. But that much code, written that fast, by one person plus AI, will have corners nobody has really exercised.
- **The licence (FSL) and the planned paid Cloud** suggest he intends to build a business around it, which is a reason to keep going but not a guarantee.

What about in the issues? Does it have other people involved? In other words, developers using it?

Used 3 tools, loaded tools

Very few. Of its **131 issues, only 4 come from outside users.** The rest were opened by Daniel himself or by bots (nightly test failures, dependency updates).

The outside ones:

- **#119 (July), wanjohiryan:** dev mode broken for SvelteKit, Nuxt and Astro. A real user hitting a real wall.
- **#193 (July), SamJB123:** asking for Postgres through Hyperdrive (PlanetScale).

- **#385 (August), cameronapak:** a database bug where deletes fail on Cloudflare's runtime with "too many terms".
- **#1232 (October), JustYannicc:** the most significant one. He's building an app called **Signalbox** and has chosen Lunora for its live-sync layer. He's offering to contribute upstream, starting with Alchemy provisioning, observability, deploy size, and trimming unused features out of deployments. His 4 commits are the only outside code.

What the rest of the issue list looks like: Daniel's own issues read like AI-run audits ("Audit 2026-10-19: bugs, security and performance findings across all packages") and detailed self-reported bugs, then get fixed fast. It's thorough, but it's one person talking to himself.

Two open issues worth knowing about if you were considering it:

- **#1166:** the more functions an app has, the more startup CPU the Worker needs, until a **large app can't deploy.** Signalbox is offering to help with exactly this.
- **#1310:** looking up a record by ID alone probes every table, costing 70 to 125 ms per call on global tables.

In short: about 290 stars, one serious adopter (Signalbox) and a couple of early experimenters. It's not yet a project with a user community. You'd be one of its first real production users.

Those last two issues are bloody serious. Can you look into it? Yeah.

Used 5 tools, ran 2 commands, loaded tools

I read both issues, the fixes merged since, and the PR descriptions behind them.

#1166: a large app can't deploy (the serious one)

The problem

- Cloudflare gives a Worker **1 second of CPU to start up**, and you can't raise it.
- Lunora's generated function registry imports **every function and everything those functions import** at startup, whether or not a request ever calls them. So startup cost grows with the size of your app.
- Their own production app (about 1,000 functions, 130 tables, an 11 MB bundle) took around 350 to 420 ms of CPU to start on a laptop, and **Cloudflare rejected the upload** because its CPUs are slower. Every Worker sharing that bundle fails too: workflows, queue consumers, all of them.

What's been done

- **A lazy registry was merged on 18 October** (PR #1236), so functions load on first use. Notably it was written by Yannic from Signalbox, not Daniel. **But it's opt-in** (`--lazy-functions`), not the default.
- By their own measurements, lazy loading cuts startup by about a third, and the theoretical best case is about half. Schema, auth and the rest still load eagerly.
- **The PR itself says the improvement is unproved.** On their synthetic 1,000-function test it was *slower and larger* locally, a benchmark showed a 29% regression (with some doubt about the cause), and it was never tested against the real app that failed.
- They've added a CI check on startup size and a pre-deploy test that loads every lazy module. Other cheaper fixes are still open, and **so is the issue.**

What it means: small and medium apps are fine. A big app runs into a hard Cloudflare wall, and the fix so far is partial and unproven.

#1310: lookups by ID are slow (a trap, not a wall)

The problem

- `ctx.db.get(id)` with no table named has to check every table, because IDs are random UUIDs that don't say which table they belong to.

- On global (D1) tables, that's one round trip across all of them. In their app, with 41 tables, that's **70 to 125 ms per call**. Three calls made one request take 0.4 to 0.5 seconds, compared with 7 to 15 ms when the table was named.

What's been done

- **Partly fixed on 23 October** (PR #1313): `ctx.db.<table>.get(id)` now checks only that table and caches the answer.
- The plain `ctx.db.get(id)` is **unchanged**, because fixing it needs an API decision. The issue stays open, and the code checker doesn't warn about it yet.
- That same fix slipped in a contract change: explicit IDs containing a NUL character are now refused.

What it means: avoidable if your code always names the table. But it's an easy mistake, and nothing stops you making it.

Overall: #1310 is a footgun with a known workaround. #1166 is a real ceiling. Lunora's design front-loads a lot of generated code into every Worker, and the fix is still opt-in and unproven. Both issues were found in Daniel's own app, which is good honesty, but it also shows nobody else has run Lunora at that scale yet.

So how quickly were these serious issues fixed?

Fast to respond, but neither is fully fixed.

#1166, the startup limit (opened 14 October)

- **Same day:** a CI check added on startup size (#1165).
- **15 October:** a written plan for the lazy registry.
- **17 October:** Yannic from Signalbox offered to build it, and opened the PR the same evening.

- **18 October:** merged, about **4 days after the issue**, as an opt-in flag.
- **19 October:** a pre-deploy test that loads every lazy module (#1257).
- **Today, 11 days on:** the issue is still open. Lazy loading isn't the default, the cheaper fixes haven't landed, and nobody has shown it actually gets the failing app under Cloudflare's limit.

#1310, slow lookups by ID (opened 22 October, 23:26)

- **19 minutes later:** a fix PR opened.
- **23 October, 09:34:** merged, about **10 hours after the issue.**
- **But it only fixes the table-named path.** The plain `ctx.db.get(id)` is still slow, and the issue is still open pending an API decision.

The pattern: reactions are very quick, within hours or days, because it's one person with AI agents and a release bot shipping several alphas a day. Several other bugs filed the same night were closed within a day too.

The flip side is that **quick partial fixes get merged and the hard part stays open.** Both PRs say so honestly in their own descriptions. The startup fix arrived through an outside contributor's opt-in change that its own author calls unproven. So expect fast patches, but not necessarily fast resolutions.

Can you give me the URLs to those two issues?

- #1166, startup limit: https://github.com/anolilab/lunora/issues/1166
- #1310, slow lookups by ID: https://github.com/anolilab/lunora/issues/1310

And the two fix PRs:

- #1236, lazy registry: https://github.com/anolilab/lunora/pull/1236

- #1313, table-named lookup fix: https://github.com/anolilab/lunora/pull/1313

https://github.com/anolilab/neore/blob/main/AGENTS.md

Maybe you can learn more about the platform from this and its links

Read 4 files, ran 2 commands, used a tool

This is the most revealing thing yet. **Neore is Daniel's own product**: Neore Chat v2 (neore.ai), an AI chat app, and the "real app" behind both of those issues. Its AGENTS.md is a ~2,000-line diary of what actually happened building on Lunora.

What Neore is

- A React (TanStack Start) web app, a browser extension, and a Tauri desktop and mobile shell.
- A Lunora backend: ShardDO plus global D1 tables plus R2, with Better Auth.
- Five separate Hono Workers: LLM gateway, document parser (Rust), browser renderer, embeddings, NSFW checker.
- **Deployed with Alchemy 2.0 beta, not `lunora deploy`.** Production, staging and preview each live in their own Cloudflare account.
- Its own heading reads "LUNORA (CODE COMPLETE — AWAITING FIRST DEPLOY)". Staging deploys have happened, but production hasn't launched yet.

The biggest revelation: sharding doesn't work the way Lunora's docs suggest

The docs say a `.shardBy()` table routes every call by that field. Neore's notes say plainly that `ctx.db` **never routes by** `.shardBy()`**, and a row lands in whichever DO ran the write.** So Neore built its own per-user sharding on top: helpers to send each request to the caller's shard,

to cross into another user's shard, to look up who owns a webhook, and to fan the cron out to every user's shard. It even had to wrap Lunora's scheduler, because jobs scheduled on a user's shard ran on the root shard and found none of that user's data.

Problems they hit for real (all type-checked clean, all failed on the first real request):

- **Every new isolate's first global-table access costs about 3 seconds**, because Lunora creates tables lazily (368 statements). From a deployed Worker that took **a full minute** until they batched it.
- **Concurrent first-page loads on one shard stacked up:** 5 queries took 4.9 seconds, 10 took 8.8 seconds, and then **the backend died.** They now budget first-paint queries and test that budget.
- **One unbounded outbound `fetch` in the root shard blocked everything**, because a DO handles one request at a time. Every outbound call now needs a deadline.
- **No cron ever ran once deployed**, because the Worker entry didn't forward `scheduled`.
- The web Worker sits near Cloudflare's 64 MiB size cap (about 3,300 chunks), and then there's the startup CPU wall from #1166.
- A long list of small traps (global tables rejecting the older query builder, `findMany` with no limit returning every row, `patch` refusing `undefined`) and **generated code that silently degrades to `unknown` while codegen exits 0.**
- They pin dozens of upstream issues and a strict process for bumping Lunora's alpha versions, because a meta-package lags what it pins.

What this means

- **Credit where due:** this is very honest and very thorough. The same person is using it in anger and fixing what breaks.
- **But the only real app on Lunora is its author's, it isn't in production yet, and it needed a lot of custom code around sharding**, which is exactly the piece we care about for per-country.
- So Lunora is worth mining for ideas, and Neore's AGENTS.md is the best list of Cloudflare DO and D1 pitfalls I've seen anywhere. As the foundation for your apps, it's too early.

Source: neore AGENTS.md

Christ, does this guy really know what he's doing as an architect? Because you're showing a lot of other aspects.

He clearly knows Cloudflare deeply. What looks weak is the architecture's priorities, not the engineering skill.

Signs of real skill

- **He understands Durable Objects properly.** The replica design (change log, read-your-own-writes cursor, failing safe to the owner, handling forks and trimmed logs) is subtle and correct. That's not beginner work.
- **Sensible security defaults.** Shard access is denied unless you allow it, and a jurisdiction that can't be honoured fails closed rather than leaking data out of region.
- **He measures and writes it down.** Real numbers, tests that pin each lesson, and an unusually honest record of what broke.

Signs of weak architecture

- **The default is the anti-pattern.** New apps start with everything in one root DO, Cloudflare's own warning case. His own app hit it (10 concurrent first-page queries killed the backend) and had to retrofit per-user sharding by hand.
- **The core claim doesn't hold in his own app.** The docs say `.shardBy()` routes data automatically, and Neore says it doesn't. That's the central scaling feature.
- **Predictable Workers limits hit late.** Cloudflare's 1-second startup limit and 64 MiB size cap are well known. Yet the design statically imports every function, and creates global tables lazily with hundreds of statements inside live requests. Both had to be patched after they bit.
- **Breadth before the core is proven.** 59 packages (payments, mail, voice, AI, x402 micropayments, SDKs in eight languages) while sharding, startup and the first production

deploy are still unsettled.

- **AI-driven volume.** About 190,000 lines in five months, mostly written with AI agents. Every piece gets polished locally, but nobody steps back and simplifies the whole. Generated code silently degrading to `unknown` is a symptom of exactly that.

My read: an excellent engineer working bottom-up, fixing each problem carefully as it appears, but without a strong top-down design built around Cloudflare's constraints from day one. That's the opposite of what you'd want underneath per-country apps, where the core design has to be right from the start.

One caveat: I'm judging from his docs, issues and code structure, not from running it. But his own notes are the strongest evidence, and they point the same way.

Are there any issues on the demo repo?

Used 4 tools

Yes. I assume you mean **Neore** (anolilab/neore), the app built on Lunora. It has only 10 issues and 13 stars, and they're telling.

Who's filing: Daniel, bots, and one outsider, `georgeatparallel`, who's from Parallel (a search vendor) asking to add their search tool. That's a vendor pitch, not a user. **There are no users filing anything.**

The open ones that matter

- **#307: the full test run has been failing since 22 October**, and it was still failing on pushes on the 23rd and 24th, plus the nightly runs.
- **#297, an AI-run audit on 20 October** across every package. Its own findings include:

- **A top-priority billing bug: LLM usage on the platform's keys charged users nothing.** Pricing lookups read the models.dev data in the wrong shape, so every cost came out as zero.
- **"Stop generating" left every token before the cancel unbilled.**
- **GDPR data export downloads returned 404**, and a failed export blocked new requests for 30 days.
- **The LLM gateway ran in development mode on public staging and preview URLs**, exposing stack traces, Swagger UI and a mock-LLM switch.
- Paid image, audio and transcription workflow steps with no daily quota and no guest block.
- **No unit tests at all** on the auth, organisation, team, API-key, billing and checkout modules, and almost none on billing in the web app.
- "God modules", including a 6,262-line `chat/functions.ts`.
- It notes fix branches were in flight, but "nothing has been merged" at the time of writing.

The background: the repo dates from June 2025, older than Lunora, so Neore was migrated onto it. It still hasn't launched in production.

The picture this adds to: most of these are app bugs, not Lunora bugs. But they reinforce the same pattern: very fast, AI-driven, broad development with thorough self-audits, and serious gaps in the fundamentals (billing, tests, a green build) in the one app that's meant to prove the platform.

Lunora

You said there were users using it. Is there any way to see their GitHub repos that are using this?

Used 12 tools, ran a command

Yes. I searched all public GitHub code for Lunora imports outside anolilab, and **only two projects use it.**

1. cameronapak/dotflowy: tried it, then retired it

- An open-source Workflowy alternative, live at dotflowy.com, with 87 stars. The author is the cameronapak who filed issue #385.
- **July 2026:** adopted Lunora as an opt-in beta for syncing outlines, so he could stop maintaining his own sync engine.
- **16 September to 2 October:** a PR titled **"Retire Lunora through per-user cutover"**. It moves every user back to his original hand-built DO, keeps the Lunora data read-only for a 30-day observation period, and his AGENTS.md now says user-facing copy must not even mention Lunora.
- His decision record doesn't say *why*, but his earlier design notes record serious traps he hit:
  - **A row disappeared for one moment after a confirmed write**, so "Couldn't open today's daily note" appeared for every user, once a day.
  - **A race that could silently overwrite another device's data**, which he could narrow but not close.
  - IDs resolving across tables unless the table was named, the same family as #1310.

2. JustYannicc/signalbox: still integrating

- A fork of T3 Code (an app for agents, chats and tasks), created on 2 October, with 1 star and 502 open issues.
- Lunora sits in its `packages/sync`, with adapters, proof-of-concept tests and startup measurements. The author is the contributor who wrote the lazy-loading fix.

The other people who filed issues (wanjohiryan, SamJB123) have no public code using it.

Bottom line: one serious outside adopter shipped it to real users and then backed out. The other is still mid-integration on a brand-new fork. Private use wouldn't show up in this search, but there's no visible public evidence of anyone running Lunora successfully in production.

So what does Signal Box do?

GitHub: Get file contents

Signalbox is a control panel for the AI coding agents on your computer. It's a fork of T3 Code, the open-source "agent harness control surface" from Theo's ping.gg team.

What it does

- **Drives the coding agents you already pay for:** Claude Code, Codex, Cursor, Grok Build, OpenCode and Google Antigravity. If one is installed and logged in on your machine, Signalbox can run it.
- **From desktop, web or phone:** a server runs on your computer (port 4783), and you control your agents from a desktop app, a local web app, or remotely from your phone or another machine.
- It also has permission modes, multiple accounts per provider, source-control integrations, connections to outside agents over MCP, and a background service.
- It installs alongside T3 Code without sharing its data (it keeps everything in `~/.signalbox`).

Where Lunora comes in

The agents and their sessions run locally. Lunora is for a **cloud sync layer** (`apps/cloud` and `packages/sync`), so your sidebar and thread lists stay live across your devices. By the author's own description in the Lunora issue, they're starting small: the sidebar and the thread list for one folder.

Status: created on 2 October, 1 star, no releases yet (the README says to build from source until the first one), and 502 open issues, mostly its own planning. So it's an early-stage project, and its Lunora use is still at the proof-of-concept stage.

Is it complicated? You know what I mean. Yeah.

Ran 2 commands

Yes, very, in two different ways.

1. Signalbox itself is huge. The cloud side alone (`apps/cloud`) is about **475,000 lines of TypeScript.** That includes vendored pieces like a Node VM interpreter and filesystem shims, a thread engine and runner, containers, and around 25 separate "proof" experiments (Codex turns, Claude turns, kernels, isolates, run stores and so on). It's built on Effect and deployed with Alchemy. A big, ambitious, very AI-driven codebase, a month old.

2. The Lunora part is deliberately walled off. They don't use Lunora directly in the app. They wrapped it in **their own sync layer** (`packages/sync`):

- About 3,200 lines of source plus 4,000 lines of tests, written in Effect.
- **13 adapter files, about 2,500 lines, that exist purely to translate between their sync design and Lunora:** client, schema, tables, validators, wire encoding, receipts, recovery and so on.
- Separate proof suites to check that Lunora's code generation and crash recovery behave as they expect.

What that tells you: a capable team that chose Lunora still didn't trust it enough to build on it directly. They put 2,500 lines of insulation between their app and Lunora, so they can swap it out. That's sensible engineering on their part, and it's also exactly what Dotflowy ended up needing when it backed out.

So for you: using Lunora "properly", the way serious adopters do, means adopting Lunora *and* building an abstraction layer around it. That's a lot of complexity to take on for an alpha platform maintained by one person.

So it sounds like they're all too complicated for trying to do what we want.

Agreed. Every one of them solves a much bigger problem than ours, and drags that complexity along: Lunora is a whole platform, fokosdb is a distributed database, Authrim is an identity system, and Signalbox and Neore are huge apps.

What we want is much simpler, and Cloudflare already provides almost all of it directly:

- **One D1 database per country**, created with the right location. That's a Cloudflare feature, not a framework.
- **Read replicas** through D1's Sessions API. Built in and free.
- **A small helper** that maps the country in the URL to the right database.
- **DOs only where live coordination is needed**, written by Cloudflare's own rules.

That's issue #5 as it stands. It stays small, uses Cloudflare's primitives directly, and has no alpha dependency underneath it.

What's still worth taking from today's digging, as patterns rather than dependencies:

- **Authrim**: how to create D1 databases through the API with a location or jurisdiction, turn on replication, and wire up the bindings without hand-editing config.
- **emdash**: clean, current D1 Sessions and bookmark code.
- **fokosdb and partyserver**: what correct DO code looks like (RPC, SQLite, retries, alarms, hibernation, tests).
- **Neore's AGENTS.md**: a real-world list of traps to avoid, like slow first access, one-at-a-time DOs blocking on outbound calls, the startup CPU limit and cron forwarding.

- **Lunora's replica design:** an idea in reserve, if we ever need DO read copies.

My suggestion: I add a short "lessons from reference repos" section to issue #5 with those links and traps, so they're on hand when we build Phase 1. Shall I?

Authoring. Let's dig into that one.

Authrim

Ran 7 commands, used 2 tools

Yeah, let me know how it meets our goals of writing the same thing in multiple Cloudflare places, if I can use that loose term.

Ran a command

Authrim matches our goal more closely than anything else we've looked at, though it splits by **tenant** where we want to split by **country**. Here's how it does "the same thing in multiple Cloudflare places".

One codebase, deployed once. The same set of Workers serves everyone. What changes per tenant is *where that tenant's data lives*.

1. D1 databases placed by region and created automatically

- A control plane creates each D1 database through Cloudflare's API with a **location hint** (`apac`, `weur` and so on) or a **jurisdiction** (EU or FedRAMP), and **turns on read replication.**
- It records each database in a registry, then **patches the bindings onto the running Workers through the API**, creates a new deployment, runs a signed smoke test, and **rolls back automatically if the test fails.** That neatly solves the problem that D1 bindings are normally fixed at deploy time.
- It keeps core identity data and personal data in **separate databases**, so residency rules can treat them differently.
- Routing **fails closed:** a missing or mismatched route errors out rather than falling back to a shared database.
- Creation is a careful state machine: it classifies which Cloudflare errors are permanent and which are retryable, retries with backoff for up to two hours, and checkpoints so a crash never creates duplicates.

2. Durable Objects placed by region ("generation-based region sharding")

- Used for sessions, auth codes and challenges.
- **The region and shard are built into every ID**, for example `default:apac:s:3`. Any Worker can read the ID and go straight to the right DO in the right region, with no lookup.
- The number of shards and the regional mix are set per tenant in KV. Changing them starts a **new "generation"** while existing IDs keep working (it keeps up to five old generations). That's online resharding, which Lunora can't do.
- It applies jurisdiction namespaces and location hints when creating DOs.

Against our goals

Our goal	Authrim
Same code everywhere	✅
Data homed in a region	✅ D1 and DOs, by location hint or jurisdiction
Local reads	✅ D1 replication (DOs have none)
Created automatically	✅ The best we've seen
Chosen by country from the URL	❌ Chosen per tenant by an operator
Simple	❌ About 900,000 lines of TypeScript, and the binding patcher alone is 944 lines

Health: essentially one developer (Yuta Hoshina) working with Claude. It's very active (commits today on `develop`, monthly releases to `main`), pre-1.0, without a security audit, with 21 stars, and all but one of its 18 issues are his own.

What to borrow for issue #5, as ideas rather than code:

1. **Put the country in the ID** (`th:venue-123`), so any Worker routes straight to the right database or DO with no lookup.
2. **Its D1 creation state machine:** classifying errors, retrying with backoff, never double-creating.
3. **Fail-closed routing:** an unknown country errors out, and nothing ever falls back to a shared database.
4. **Generations,** if we ever need to split a big country later without breaking existing IDs.

One difference works in our favour: our country list is fixed and known, so we can generate the D1 bindings at build time (as issue #5 plans) and skip Authrim's live binding patcher entirely.